

Experiment No.	3
Experiment Title	Regression Analysis using Linear and Regularized Models ¹
Student Name	Isha
Register Number	3122247001023
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	1/08/26

1 Objective

In this experiment the goal is to build and compare four regression models, plain Linear Regression ,and the three regularized models Ridge, Lasso and Elastic Net for the task of predicting the loan amount , that a bank actually sanctions for an applicant. The experiment requires us to do more than just report a single accuracy number. We have to build the models, tune the regularization strength for each model, watch the behavior of training error and validation error as we vary the strength, and discuss overfitting, underfitting and the bias-variance trade off.

2 Problem Statement

The objective is to predict Loan Sanction Amount (USD), a continuous value, based on an applicant's demographic, income, employment, credit and property details. No, it's a regression problem, not a classification problem. Input is a mixture, of numeric and categorical applicant/property features . Output is a single predicted dollar amount . Goal is to see how a plain linear model compares against its regularised counterparts on real , fairly large scale loan data .

3 Dataset Description

Dataset Name	Predict Loan Amount Data
Dataset Source	Kaggle (originally a HackerEarth challenge), https://www.kaggle.com/datasets/phileinsophos/predict-loan-amount-data
Number of Samples	30,000 loan applications (29,660 after dropping the 340 rows with a missing target)
Number of Features	24 columns originally; 21 predictor columns after dropping identifiers, expanding to about 44 columns after one hot encoding
Number of Classes	Not applicable, this is a regresion task with a continuous target (Loan Sanction Amount)
Missing Values	10 columns had missing values after fixing a -999 sentinel in No. of Defaults. Type of Employment was the worst affected (about a quarter of rows)
Train-Test Split	80:20

4 Exploratory Data Analysis

This section covers the dataset overview, missing value analysis, target and feature distributions, and the correlation matrix, since a class distribution plot does not apply here (the target is a continuous amount, not a category).

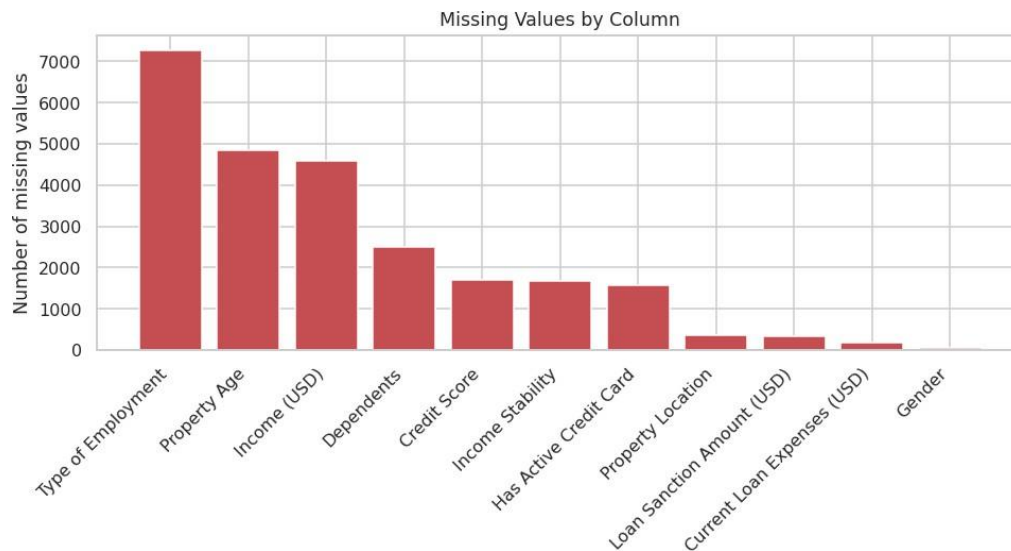


Figure 1: Missing value counts per column after fixing the -999 sentinel in No. of Defaults

Inference: Type of Employment has by far the most missing values, followed by Property Age, Income and Dependents. This is why median/mode, imputation was used instead of just dropping incomplete rows, dropping every row with a missing value would have thrown away a huge chunk of the dataset given how widespread the gaps are.

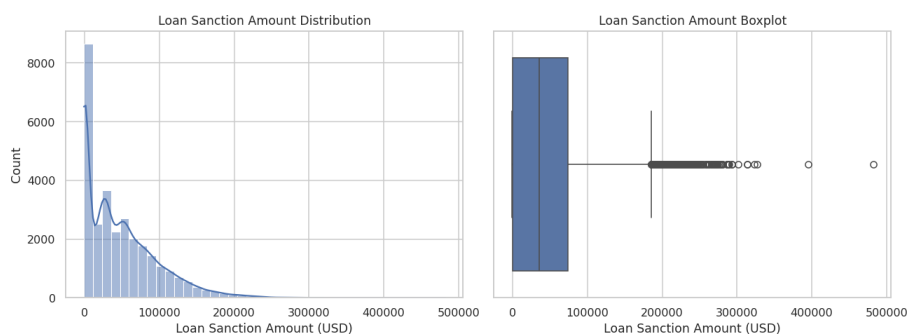


Figure 2: Distribution of the target variable, Loan Sanction Amount

Inference: the target is heavily right skewed, most sanctioned amounts sit on the smaller side with a long tail of much larger loans, and the boxplot shows plenty of points beyond the upper whisker. This matters later since a handful of very large loans can pull a linear model's fit and inflate MSE more than MAE.

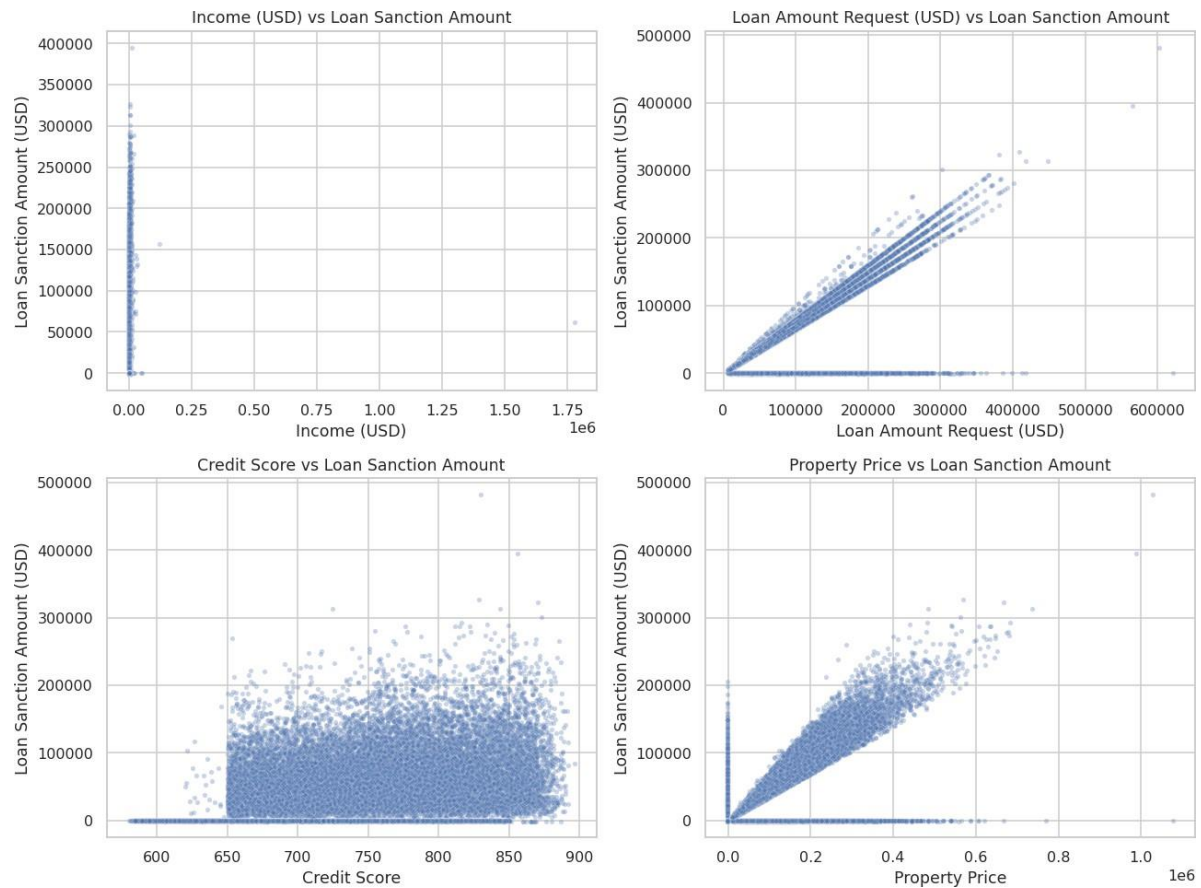


Figure 3: Feature vs target scatter plots for the main numeric predictors

Inference: Loan Amount Request and Property Price both track the sanctioned amount closely, which makes sense, the amount sanctioned should be close to what was requested and to the value of the property backing it. Credit Score shows a milder positive trend. Income on its own looks surprisingly weak and scattered, a bit counterintuitive since it likely interacts with other features instead of explaining the target on its own.

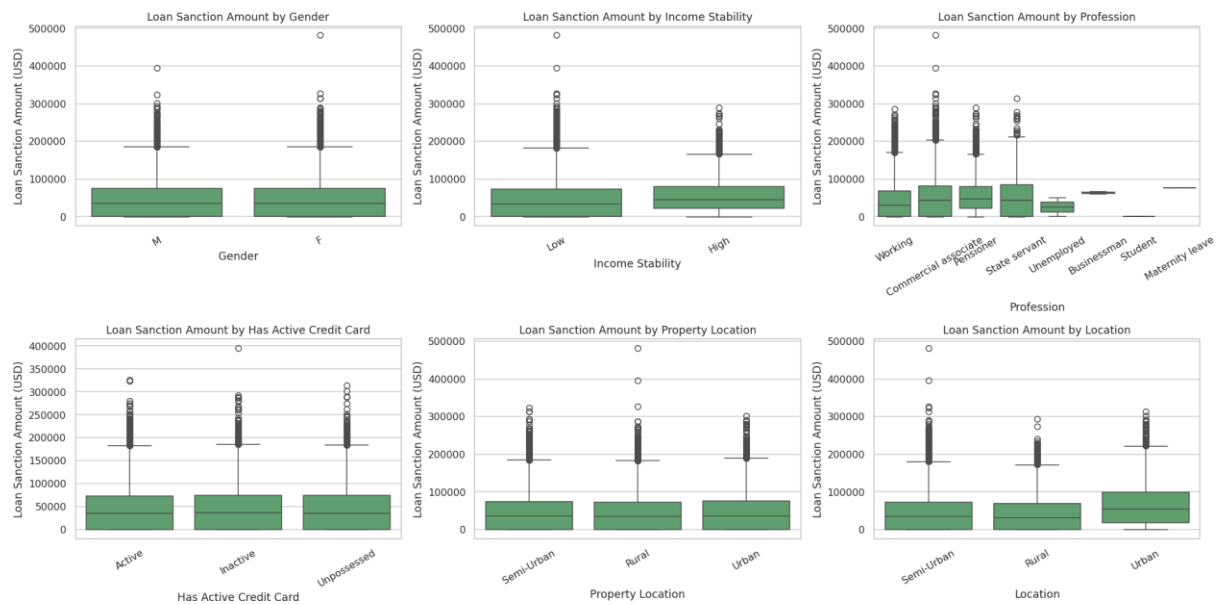


Figure 4: Categorical feature vs target box plots

Inference: none of the categorical features show, a dramatic split between categories. Profesion shows a bit more spread, professions like Businessman lean toward higher sanctioned amounts than Student or unemployed, matching knowledge about repayment capacity, but overall these features play a smaller, supporting role next to the numeric predictors above.

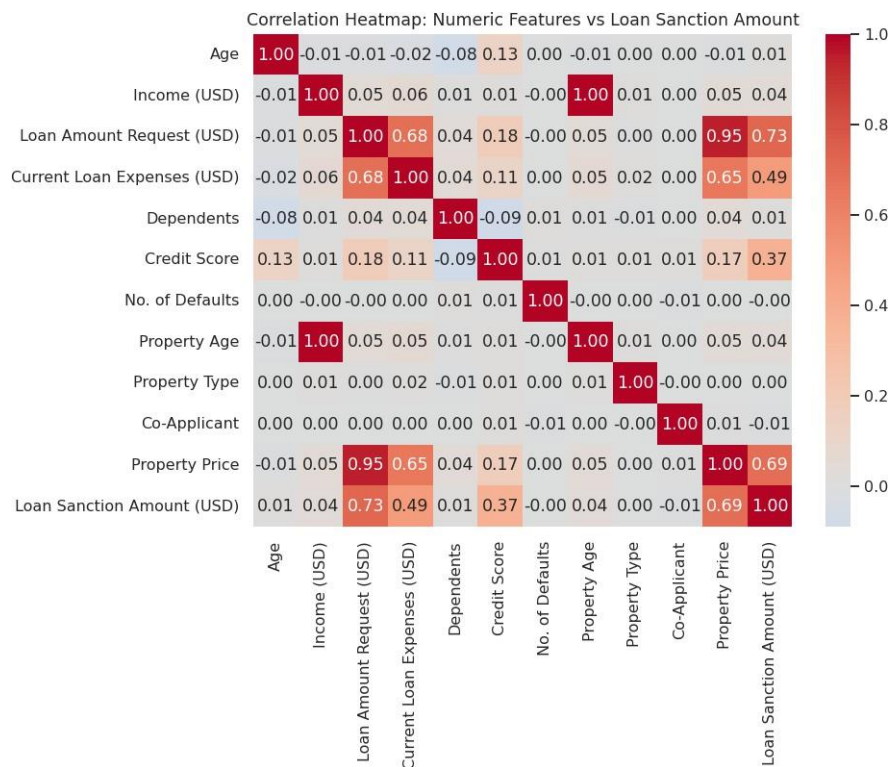


Figure 5: Correlation heatmap of numeric features

Inference: Loan Amount Request has the strongest correlation with the target

(around 0.73), then Property Price (around 0.69) and Current Loan Expenses (around 0.49). Credit Score is moderately correlated (around 0.37). Age, Property Type, Co-Applicant, Dependents and No. of Defaults sit close to zero, meaning they carry very little linear signal about the sanctioned amount by themselves.

5 Data Preprocessing

Missing value handling: the hidden sentinel, of -999 in No. of Defaults was first replaced with a real NaN. Then, the numeric predictor columns were imputed with the median of the training set (median was chosen over mean because there are quite a few outliers in the data) and the categorical columns, were imputed with the mode of training set. Since there is nothing that a regression model can learn from without a target value, the 340 rows missing the target itself were dropped completely.

Encoding: `pd.get dummies` was used to one-hot encode the nine categorical columns (Gender, Profession, Type of Employment, Location and so on), dropping one level for each category to avoid duplication. This led to the expansion, of the categorical columns to around 33 dummy columns.

Feature scaling: All features, including the dummy columns, were standardized to have mean 0 and standard deviation 1 using `StandardScaler`, fit only on the training data. This is especially important for Ridge, Lasso and Elastic Net which penalise the absolute size of a coefficient. Without scaling, an attribute like Loan Amount Request (in the hundreds of thousands) would be treated differently from a 0/1 dummy column simply due to scale, not importance.

Train-test split: the split (80:20) was done prior to imputing or scaling, so the median, mode and scaling values are learnt only on the training data. Doing it the other way around would leak test set information into pre-processing.

Feature engineering: no additional engineered features were created beyond the one-hot dummy columns; the focus of this experiment was on the regression models and regularization rather than feature engineering.

6 Machine Learning Algorithm

Linear Regression. Working principle: fits a straight line relationship between the input features and the continuous target by minimizing the sum of squared errors. Mathematical formulation:

$$\hat{y} = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

Advantages: simple, fast, interpretable, coefficients directly show each feature's effect on the target. Limitations: has no penalty term at all, so with correlated features it can assign large, unstable, canceling out coefficients that do not really reflect true feature importance.

Ridge Regression. Working principle: same as Linear Regression but adds an L2 penalty that shrinks coefficients toward zero. Mathematical formulation:

$$\text{Loss} = \sum (y_i - \hat{y}_i)^2 + \alpha \sum \theta_i^2$$

Advantages: reduces the impact of multicollinearity and keeps coefficients from growing too large. Limitations: shrinks coefficients but almost never sets them exactly to zero, so it does not simplify the model by dropping features.

Lasso Regression. Working principle: adds an L1 penalty instead of L2.

Mathematical formulation:

$$\text{Loss} = \sum (y_i - \hat{y}_i)^2 + \alpha \sum |\beta_i|$$

Advantages: can push weak feature coefficients all the way to zero, effectively performing automatic feature selection. Limitations: can be unstable when features are highly correlated, sometimes arbitrarily picking one of two similar features over the other.

Elastic Net. Working principle: blends the L1 and L2 penalties using a mixing parameter `l1_ratio`. Mathematical formulation:

$$\text{Loss} = \sum (y_i - \hat{y}_i)^2 + \alpha \left(r \sum |\beta_i| + \frac{1-r}{2} \sum \beta_i^2 \right)$$

Advantages: gets some of Lasso's feature selection along with Ridge's smoother shrinkage, useful when features are correlated but some feature selection is still wanted. Limitations: has two hyperparameters to tune instead of one, so grid search takes a bit longer.

7 Experimental Setup

Python Version	3.12
Scikit-Learn Version	1.8.0
IDE	Jupyter Notebook
Operating System	Linux (Ubuntu)
Hardware	Standard CPU, no GPU required for this experiment

8 Hyperparameter Selection

Parameter	Value
Ridge search space (α)	0.01, 0.1, 1, 10, 100
Lasso search space (α)	0.001, 0.01, 0.1, 1, 10
Elastic Net search space (α)	0.01, 0.1, 1, 10
Elastic Net search space (<code>l1_ratio</code>)	0.2, 0.5, 0.8
Search method	GridSearchCV, 5-fold cross validation, scoring = R^2
Ridge best parameters	$\alpha = 100$
Lasso best parameters	$\alpha = 10$
Elastic Net best parameters	$\alpha = 0.1$, <code>l1_ratio</code> = 0.5

9 Results

Metric (Linear Regression, Test Set)	Value
MAE	21,591.26
MSE	1.019×10^9
RMSE	31,921.27
R^2	0.5511
Training Time	0.0672 s

Note: this experiment is a regression task, so Accuracy, Precision, Recall, F1-score and ROC-AUC do not apply here, MAE, MSE, RMSE and R^2 are the equivalent regression metrics used instead. The table above shows Linear Regression's test performance as the baseline; the full side by side comparison across all four models (Linear, Ridge, Lasso, Elastic Net) is given as Tables 1 through 5 in the Conclusion section, since that is where a complete comparison is most useful to read.

10 Result Visualization

This section covers all the graphs generated to evaluate model performance and behaviour: test error comparison, predicted vs actual plots, residual plots, the training vs validation error curve, and the coefficient comparison (used here in place of a feature importance plot, since none of the four linear models has a built-in importance score).



Figure 6: Test set MAE and RMSE compared across all four models

Inference: all four models sit within a few hundred dollars of each other on both MAE and RMSE, there is no clear winner by eye, which already hints that regularization is not doing much heavy lifting on this particular dataset.

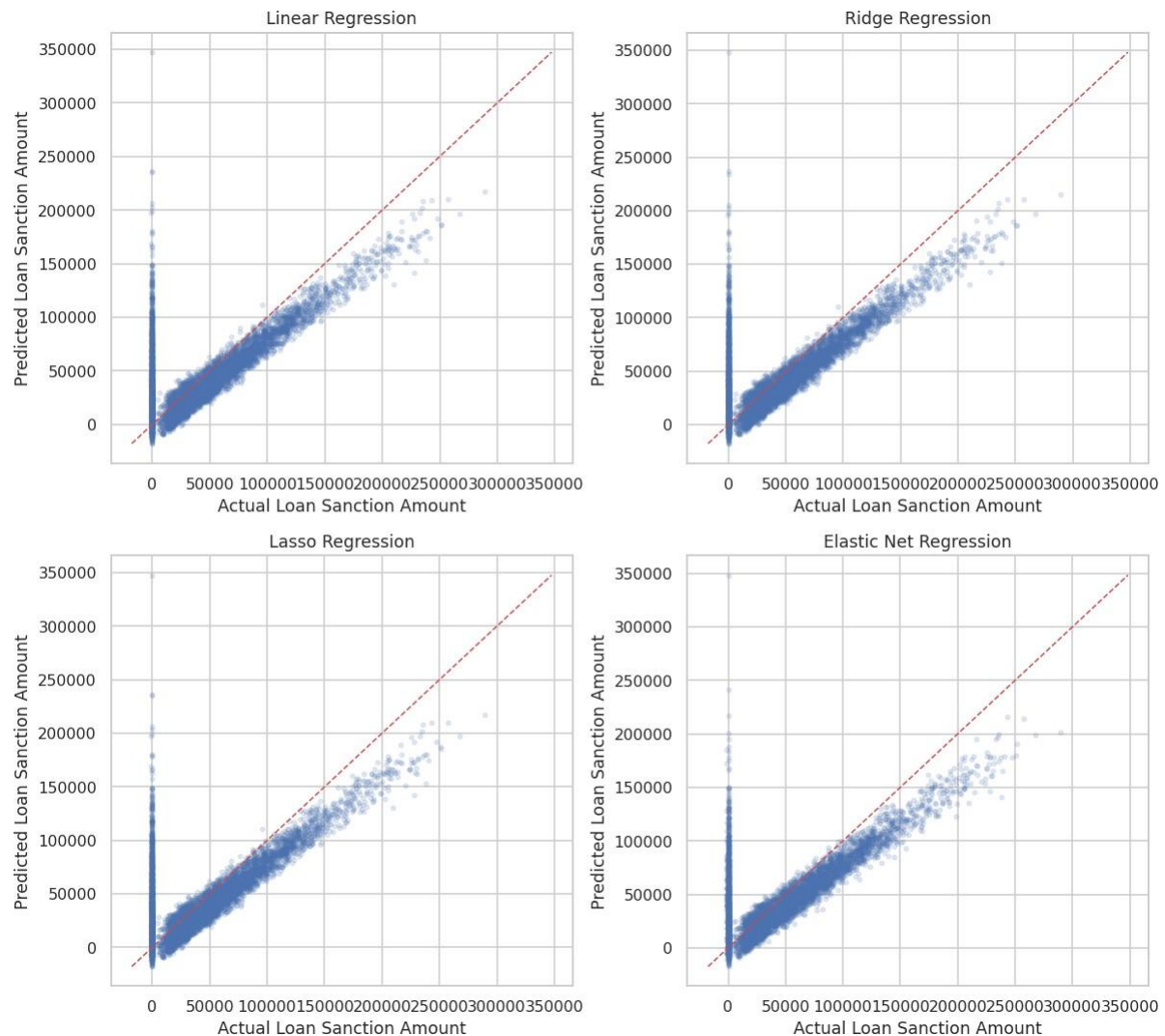


Figure 7: Predicted vs actual sanctioned loan amount for each model

Inference: points close to the red diagonal are well predicted. All four models track the diagonal tightly for small to mid sized loans, where most of the data sits, and spread out more for the largest loans, where there are fewer examples to learn from. The four plots look nearly identical, confirming there is no meaningful difference in error pattern between the models.

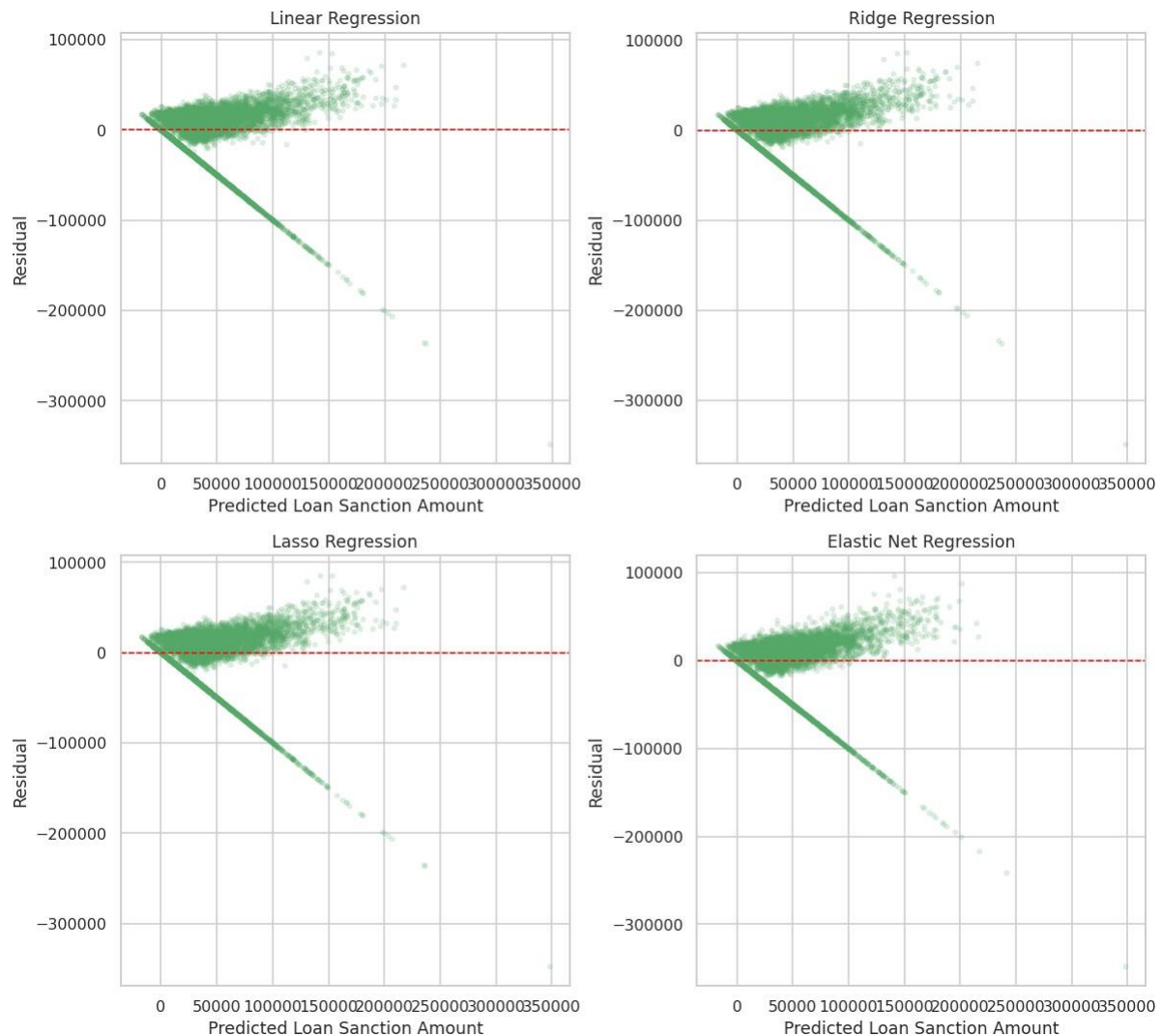


Figure 8: Residual plots for each model

Inference: the residuals fan out into a funnel shape as the predicted amount grows, a sign of heteroscedasticity, the model is less consistent on the small number of very large loans than on the many small to mid sized ones. A log transform of the target is a common fix for this exact pattern, though it was left out here so error metrics stay in the original dollar scale as the manual asks.

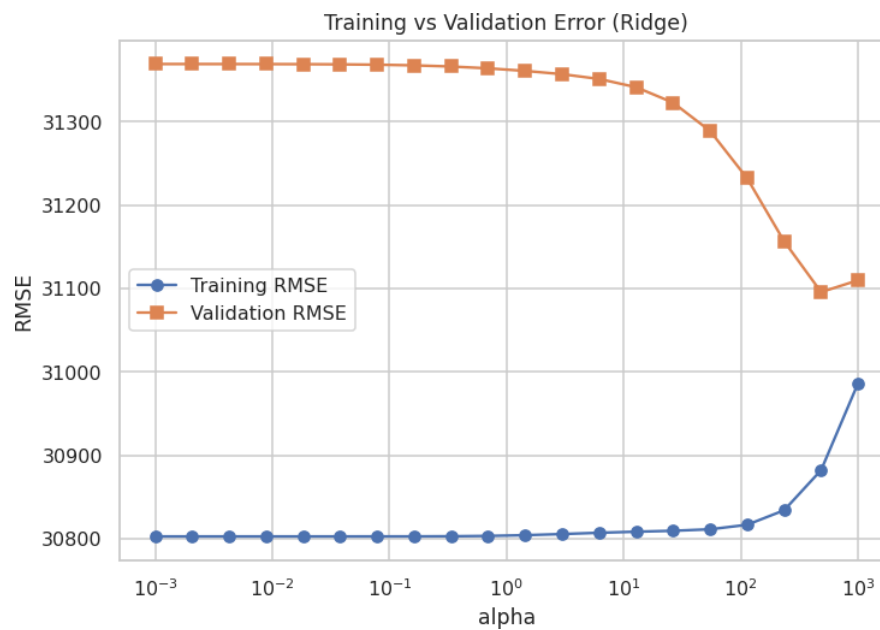


Figure 9: Training vs validation RMSE across a range of Ridge alpha values (Learning Curve style analysis)

Inference: For small alpha, both the training and validation error lie close to each other and are quite low to start with. As the value of alpha approaches infinity, both curves tend to increase together in a typical case of underfitting. In this scenario, the regularization is sufficiently strong enough to remove true information, along with the noise from the data.

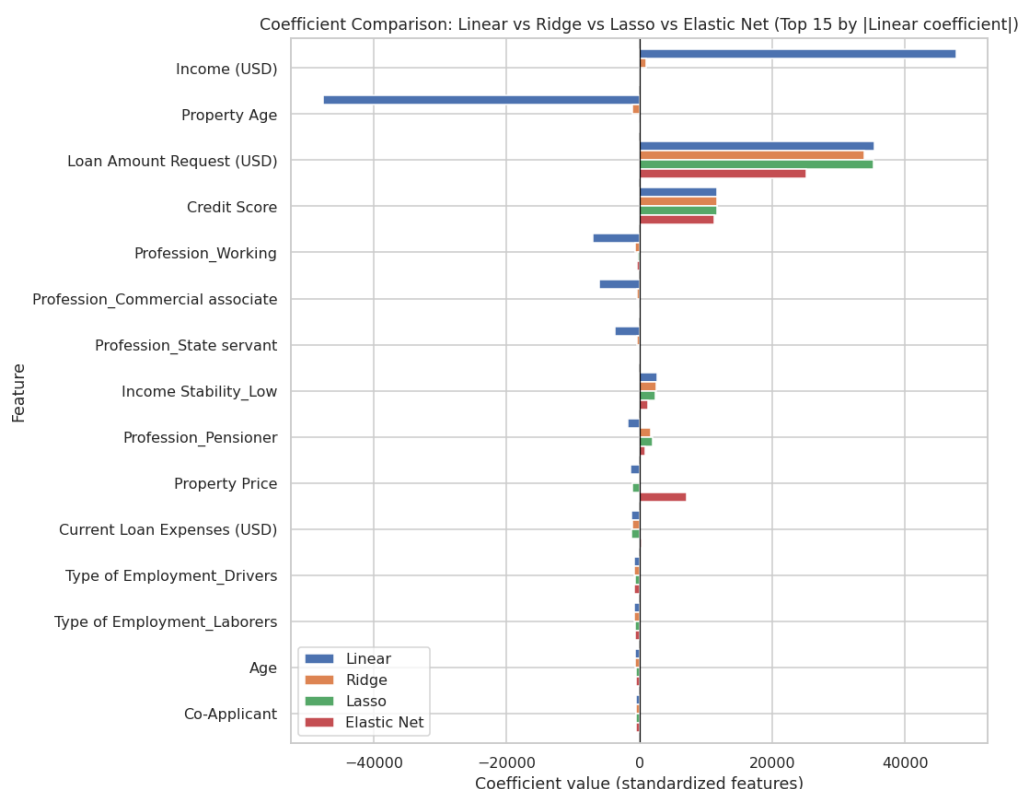


Figure 10: Coefficient comparison across Linear, Ridge, Lasso and Elastic Net (Feature Importance, top 15 by magnitude)

Inference: However, when we look at their exact values (Table 4, Conclusion section), we can see how Income and Property Age end up with very high, almost inverse, coefficients using basic Linear Regression, which is a symptom of multi-collinearity, but Ridge regression reduces their value significantly, and Lasso makes Property Age equal to zero. Loan Amount Request and Credit Score remain high and constant in all regression types.

11 Discussion

Overfitting and underfitting: The training and test errors were almost identical in all the four models; each of the test RMSEs was just a few hundreds away from the training RMES (Table 5, Conclusion section). Given around 23,700 training rows and roughly 44 encoded features, there were plenty of data relative to the model complexity, so that none of the four models overfitted in any meaningful way. Examining the impact of the strength of regularization, Ridge yielded its optimal alpha in grid search to be 100, the largest value in the range of its grid search; at the same time, the test performance of this model changed just negligibly from that of ordinary Linear Regression, which is consistent with the training-validation curve: shrinkage does not harm the fit much because, it was rather stable in the first place, but it does not improve the performance much because there was no overfitting. The benefit of the regularisation after tuning, was minimal for this dataset: all four models achieve almost identical test R2. The main advantage of Lasso in terms of performance was automatic selection of a few weak features.

Bias-variance trade-off. Simple Linear Regression does not have a penalty term and, therefore, it is the model with the least bias among all the four models that were used because it tries to fit the data as much as possible, with the linear function. With the help of sufficient amount of training data, the variance was kept at a reasonable level for this case, however, variance could have been a problem for the case when there would be much less data to train on. Ridge and Elastic Net introduce an L2 regularization term into the equation and, therefore, do not allow any of the coefficients to increase dramatically due to the presence of some outlier group of data, although the reduction in variance for such a large dataset was minimal; that explains the proximity of the test results.

Strengths, limitations and possible improvements: The biggest advantage of this comparison is the use of a large and realistic dataset (30,000 observations), which allowed for a fair assessment of the ability of regularization to be effective when scaled up, and the conclusion was that it is effective only marginally. A weakness is the right-skewed distribution of the target variable and heteroskedasticity of the residuals, apparent from the residual plots, neither of which is addressed effectively by any of the four linear models. A log transformation of the target, or even a non-linear model like Random Forest or Gradient Boosting, could probably perform better than the ~45% of the variance left unexplained by all four models here (test $R^2 \approx 0.55$).

12 Conclusion

The current experiment employed Linear, Ridge, Lasso, and Elastic Net regression in predicting the sanctioned amount in a dataset of 30,000 loan requests. The results of the four methods were almost the same in the test set, R^2 of around 0.55 for each, indicating that with the current amount of data and 44 encoded features, the use of Linear Regression would not lead to significant overfitting and therefore, did not require any regularization. The use of Lasso was actually more effective in simplifying the model by removing redundant features rather than improving the accuracy. The tables below, obtained from the current experiment, describe all steps in the experiment according to the lab manual.

Table 1: Hyperparameter Tuning Summary

Model	Search Method	Best Parameters	Best R^2	CV
Ridge Regression	Grid Search	$\alpha = 100$	0.5816	
Lasso Regression	Grid Search	$\alpha = 10$	0.5797	
Elastic Net Regression	Grid Search	$\alpha = 0.1, l1_ratio = 0.5$	0.5857	

Table 2: Cross-Validation Performance (K = 5)

Model	MAE	MSE	RMSE	R^2
Linear Regression	21,509.61	9.848×10^8	31,381.18	0.578
Ridge Regression	21,530.23	9.765×10^8	31,248.92	0.582

Lasso Regression	21,503.96	9.807×10^8	31,316.68	0.580
Elastic Net Regression	21,720.92	9.675×10^8	31,105.38	0.586

Table 3: Test Set Performance

Model	MAE	MSE	RMSE	R^2
Linear Regression	21,591.26	1.019×10^9	31,921.27	0.5511
Ridge Regression	21,588.22	1.018×10^9	31,901.95	0.5516
Lasso Regression	21,568.23	1.018×10^9	31,912.31	0.5513
Elastic Net Regression	21,754.22	1.019×10^9	31,926.64	0.5509

Table 4: Coefficient Comparison (Top 10 Features by —Linear Coefficient—)

Since all features were standardized before fitting, coefficient magnitudes are directly comparable across models. These are the exact values printed from the notebook, sorted by the size of the plain Linear Regression coefficient.

Feature	Linear	Ridge	Lasso	Elastic Net
Income (USD)	47,689.46	1,040.05	4.85	93.42
Property Age	-47,676.04	-1,031.17	0.00	-86.44
Loan Amount Request (USD)	35,370.84	33,828.69	35,169.98	25,146.82
Credit Score	11,636.76	11,606.01	11,638.69	11,226.45
Profession_Working	-7,082.22	-711.70	-228.87	-389.63
Profession_Commercial associate	-6,104.01	-418.34	0.00	-97.66
Profession_State servant	-3,768.19	-327.23	-64.64	-97.96
Income Stability_Low	2,714.91	2,471.63	2,442.98	1,278.29
Profession_Pensioner	-1,783.54	1,711.47	1,936.21	870.10
Property Price	-1,331.97	24.54	-1,137.30	7,064.12

The Income and Property Age rows are the most telling ones in this table. Plain Linear Regression gives them enormous, nearly opposite coefficients that mostly cancel out, a classic multicollinearity symptom, while all three regularized models pull both back down to a much smaller, more believable size. Loan Amount Request and Credit Score, on the other hand, stay large and stable across every model, a better indicator that they are genuinely important predictors rather than an artifact of an unstable fit. Lasso is the most aggressive here, sending Property Age and the Commercial Associate profession dummy to exactly zero.

Table 5: Training vs Test Error (Overfitting Check)

Model	Train MAE	Train RMSE	Test MAE	Test RMSE
Linear Regression	21,436.47	30,811.10	21,591.26	31,921.27
Ridge Regression	21,459.07	30,821.50	21,588.22	31,901.95
Lasso Regression	21,436.89	30,818.41	21,568.23	31,912.31
Elastic Net Regression	21,671.98	30,984.07	21,754.22	31,926.64

13 References

1. Scikit-learn Documentation, “Linear Models,” https://scikit-learn.org/stable/modules/linear_model.html
2. Scikit-learn Documentation, “Hyperparameter Optimization,” https://scikit-learn.org/stable/modules/grid_search.html
3. Kaggle, “Predict Loan Amount Data,” <https://www.kaggle.com/datasets/phileinsophos/predict-loan-amount-data>